

Assignment 2 - Part 1: A Star Wars Process and Parallel Sums

Fall 2026

Due: Friday, Sep 18, 11:59 pm

- Download base code A2_1 from D2L

Coding Task 1: Complete the program starwars.c

- Complete the program to spawn 4 child processes each representing different Star Wars character:
 - Luke – adjusts shield power by 25%
 - Han – adjusts shield power by 20%
 - Chewbacca – adjusts shield power by 30%
 - Leia – adjusts shield power by 15%.
- Each child process should:
 - Print “<character name>: Adjusting shield power”
 - Increment the shield power by the specified percentage.
 - Print new shield level: “<character name>: Shield power level now at <x>%”.
- The parent process must:
 - Check if process creation fails, and handle errors appropriately.
 - Wait for all child processes to finish their task before it exits.

Your output should look like this.

```
Millennium Falcon: Initial shield power level: 50%

Luke: Adjusting shields...
Luke: Shield power level now at 75%
Han: Adjusting shields...
Han: Shield power level now at 70%
Chewbacca: Adjusting shield...
Chewbacca: Shield power level now at 80%
Leia: Adjusting shield...
Leia: Shield power level now at 65%

Millennium Falcon: Final shield power level: 50%

May the forks be with you!
```

Task 2: Explain the following about the program starwars.c:

- 1) Why does the initial and final value of the shield power remain unchanged?
- 2) Why does the order of output change every time you run the program?

Provide a clear, concise, and detailed explanation.

Coding Task 3: Convert the sequential array sum program to parallel using multiprocessing. Complete the parallel_sums.c program.

- Split the array into chunks of approximately equal size.
- Each child process should compute the sum of one chunk.
- Uses pipe() to send each child's partial sum back to the parent
- Then the parent process:
 - o Waits for all children to finish
 - o Collects each partial sum from the corresponding pipe
 - o Calculate the total sum and print the result.
- Ensure all file descriptors are closed properly in both parent and child processes

Task 4: Time both your sequential and parallel sums program for $N = 100,000,000$ and fill out the table below. Then explain your observations in detail. What advantages and challenges did you observe in multiprocessing.

Program	Sequential	1 process	2 processes	3 processes	4 processes
Time taken					

Deliverables:

- A zipped folder named A2_part1 submitted to D2L dropbox that contains:
 - o Your source code file.
 - o A pdf file for Task 2, Task 4, and a screenshot showing your code A2_p.c works as expected.

- A README.txt file
- Available on Github inside the folder where you added me as a collaborator.

Total Points (100)

- Code runs and works as expected – Task 1: 20 points
- Clear and detailed explanation – Task 2: 20 points
- Code runs and works as expected – Task 3: 40 points
- Clear and detailed benchmarking and explanation – Task 4: 15 points
- Available on GitHub: 5 points